

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, roc_curve, average_precision_score,
    precision_score, recall_score, f1_score, accuracy_score
)
```

```
In [2]: # Adjust the path for Kaggle vs local
CANDIDATE_PATHS = [
    "transactions 2.csv",
]

path = None
for p in CANDIDATE_PATHS:
    if os.path.exists(p):
        path = p
        break

assert path is not None, "transactions.csv not found. Please add it to input
print("Using file:", path)

df = pd.read_csv(path)
print(df.shape)
df.head(3)
```

Using file: transactions 2.csv
(299695, 17)

```
Out[2]:
```

	transaction_id	user_id	account_age_days	total_transactions_user	avg_amount_u
0	1	1	141	47	147
1	2	1	141	47	147
2	3	1	141	47	147

```
In [3]: # Parse datetime
df['transaction_time'] = pd.to_datetime(df['transaction_time'], errors='coer

# Basic sanity
print("Nulls per column:")
print(df.isnull().sum().sort_values(ascending=False).head(10))

# Derived helpers
df['country_match'] = (df['country'] == df['bin_country']).astype(int)
```

```

df['hour'] = df['transaction_time'].dt.hour
df['night_flag'] = df['hour'].isin([22,23,0,1,2,3,4,5]).astype(int)

# Summary
summary = {
    "rows": len(df),
    "unique_users": df['user_id'].nunique(),
    "fraud_tx": int(df['is_fraud'].sum()),
    "fraud_rate": df['is_fraud'].mean(),
    "country_match_rate": df['country_match'].mean(),
    "avg_tx_per_user": len(df) / df['user_id'].nunique()
}
summary

```

```

Nulls per column:
transaction_id      0
merchant_category   0
shipping_distance_km 0
transaction_time    0
three_ds_flag       0
cvv_result          0
avs_match           0
promo_used          0
channel             0
user_id             0
dtype: int64

```

```

Out[3]: {'rows': 299695,
        'unique_users': 6000,
        'fraud_tx': 6612,
        'fraud_rate': np.float64(0.02206243013730626),
        'country_match_rate': np.float64(0.9208194998248219),
        'avg_tx_per_user': 49.949166666666666}

```

```

In [4]: # Positive amounts only
amount = df['amount'].astype(float)
amount_pos = amount[amount > 0]
log_amount = np.log(amount_pos)

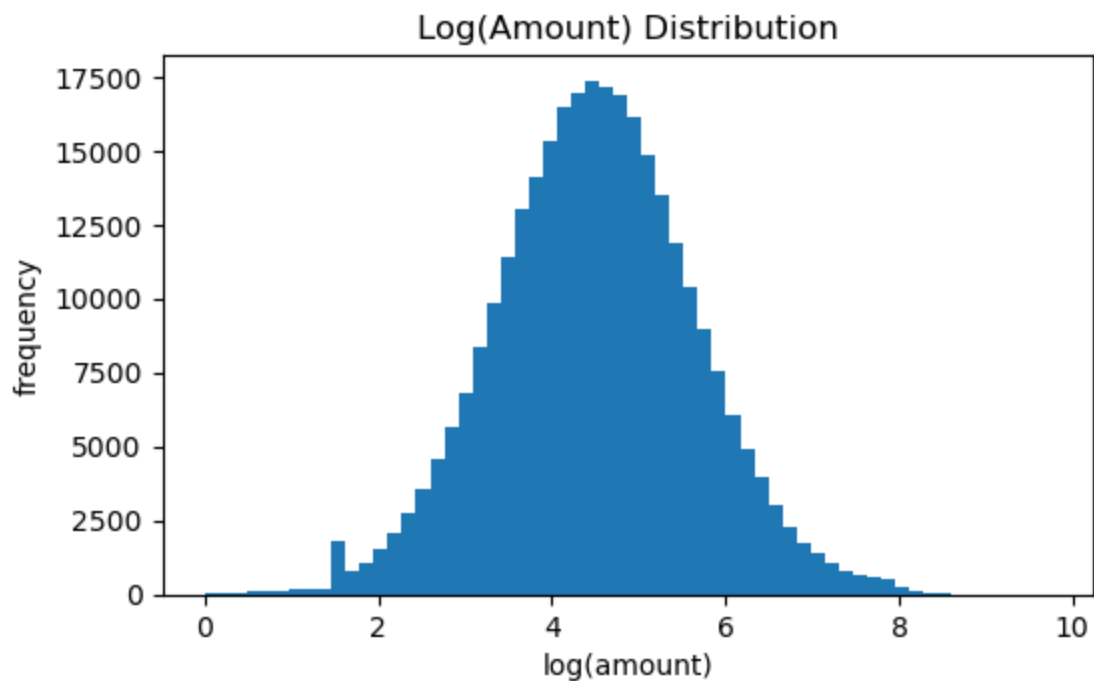
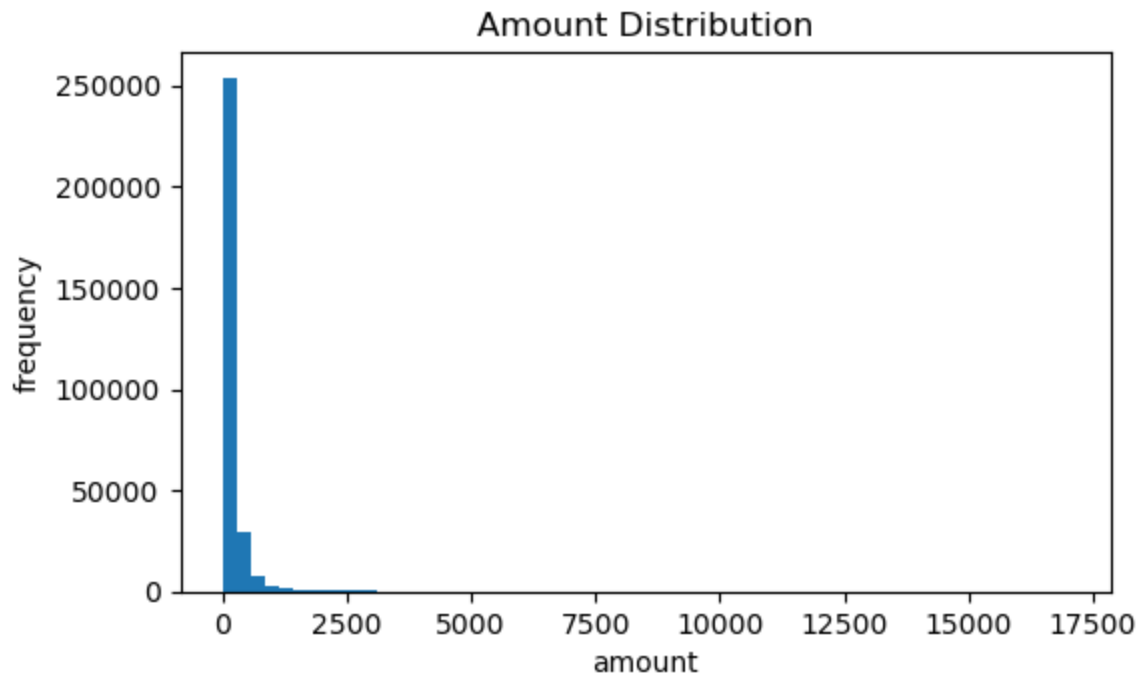
fig = plt.figure(figsize=(6,3.5))
plt.hist(amount_pos, bins=60)
plt.title('Amount Distribution')
plt.xlabel('amount')
plt.ylabel('frequency')
plt.show()

fig = plt.figure(figsize=(6,3.5))
plt.hist(log_amount, bins=60)
plt.title('Log(Amount) Distribution')
plt.xlabel('log(amount)')
plt.ylabel('frequency')
plt.show()

print({
    "amount_mean": float(amount_pos.mean()),
    "amount_std": float(amount_pos.std(ddof=1)),
    "log_amount_mean": float(log_amount.mean()),

```

```
"log_amount_std": float(log_amount.std(ddof=1)),
})
```



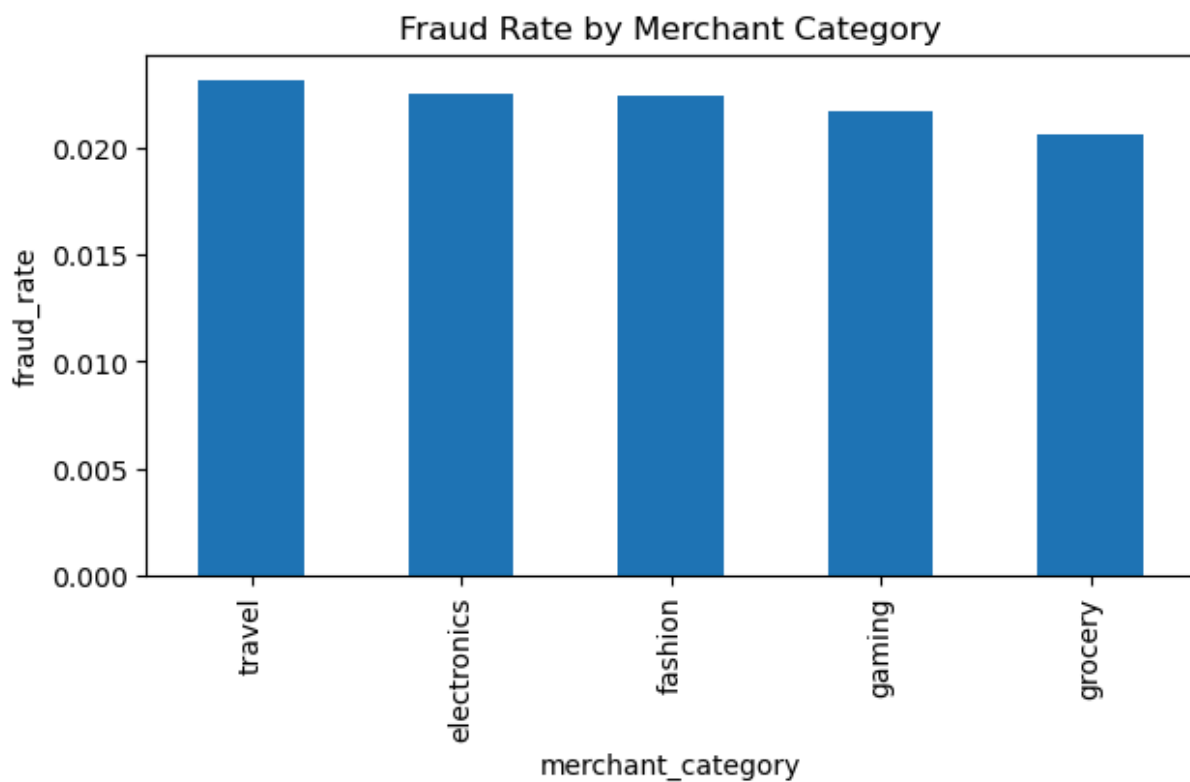
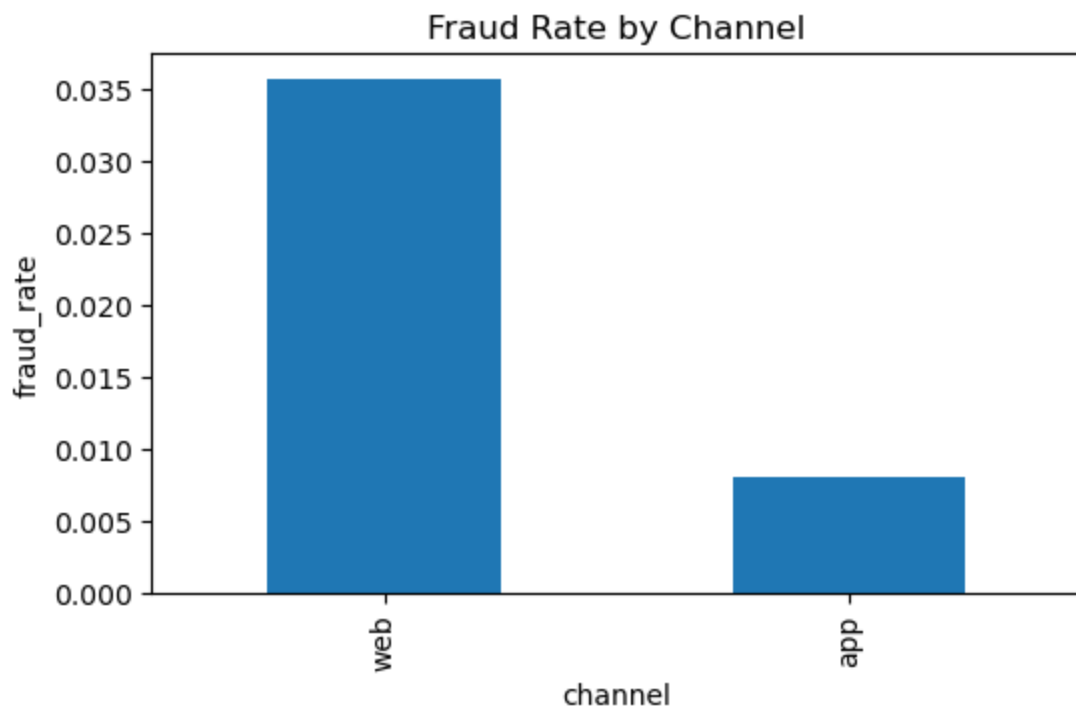
```
{'amount_mean': 177.16527856654264, 'amount_std': 306.9265069627889, 'log_amount_mean': 4.499736836509297, 'log_amount_std': 1.1526799500279332}
```

```
In [5]: fraud_by_channel = df.groupby('channel')['is_fraud'].mean().sort_values(ascending=True)
        fraud_by_category = df.groupby('merchant_category')['is_fraud'].mean().sort_values(ascending=True)

        fig = plt.figure(figsize=(6,3.5))
        fraud_by_channel.plot(kind='bar')
        plt.title('Fraud Rate by Channel')
        plt.ylabel('fraud_rate')
        plt.show()
```

```
fig = plt.figure(figsize=(7,3.5))
fraud_by_category.plot(kind='bar')
plt.title('Fraud Rate by Merchant Category')
plt.ylabel('fraud_rate')
plt.show()
```

`fraud_by_channel, fraud_by_category`



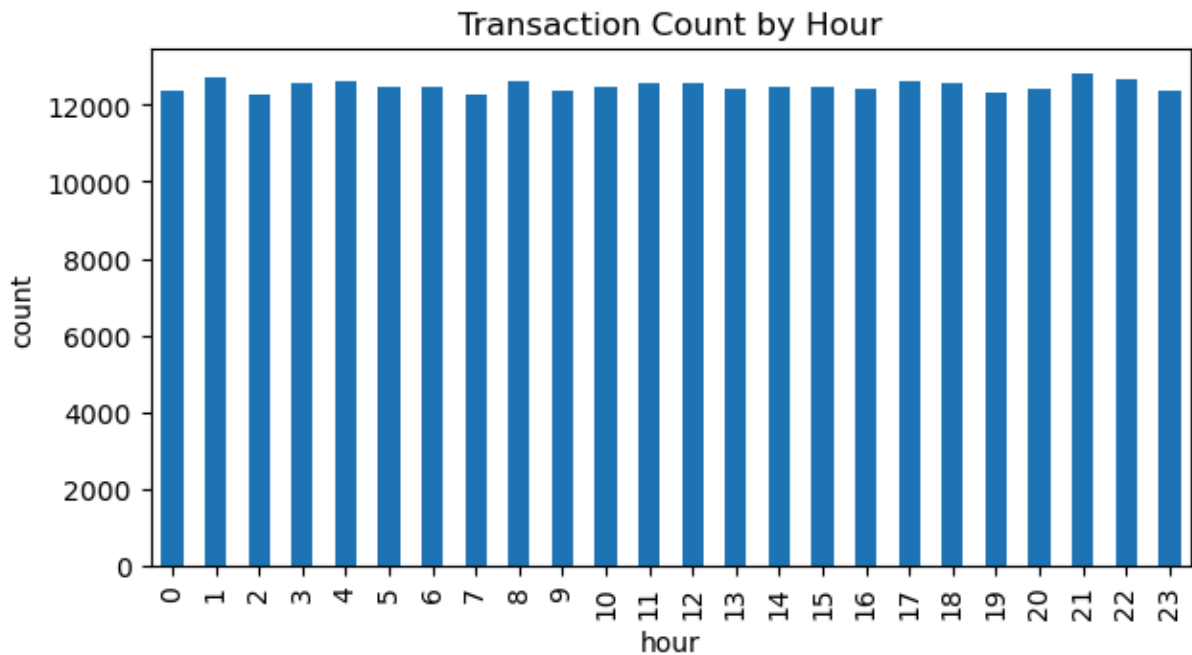
```
Out[5]: (channel
web      0.035644
app      0.008042
Name: is_fraud, dtype: float64,
merchant_category
travel      0.023163
electronics 0.022534
fashion     0.022408
gaming      0.021641
grocery     0.020563
Name: is_fraud, dtype: float64)
```

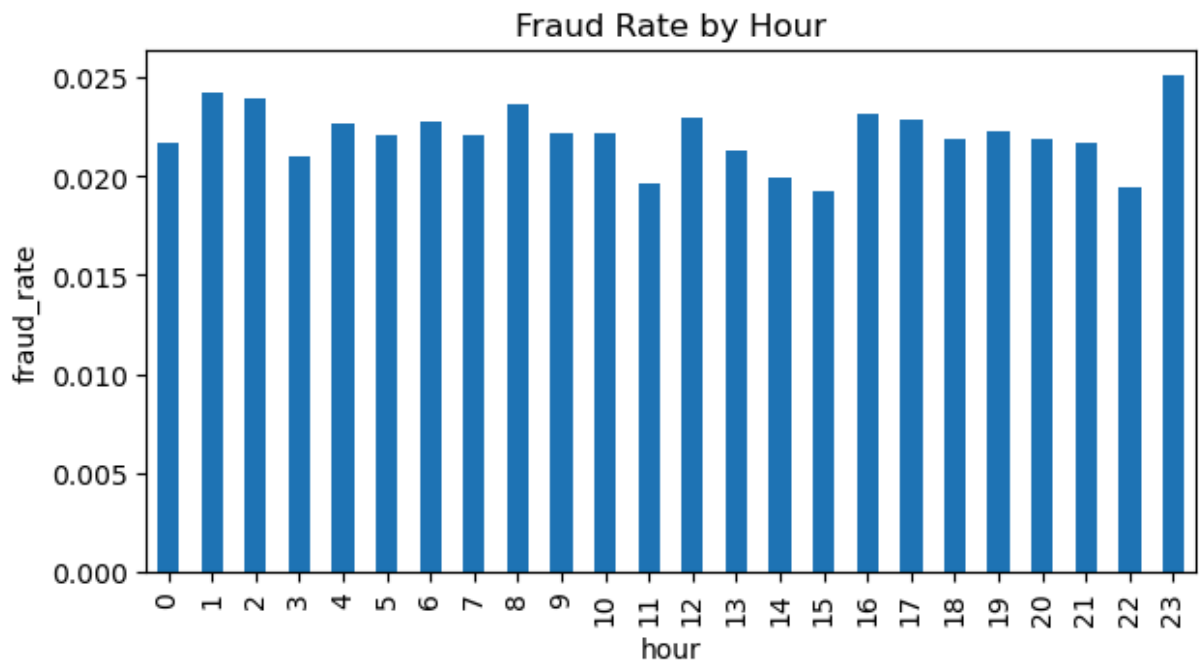
```
In [6]: fraud_by_hour = df.groupby('hour')['is_fraud'].mean()
tx_by_hour = df.groupby('hour')['transaction_id'].count()

fig = plt.figure(figsize=(7,3.5))
tx_by_hour.plot(kind='bar')
plt.title('Transaction Count by Hour')
plt.ylabel('count')
plt.show()

fig = plt.figure(figsize=(7,3.5))
fraud_by_hour.plot(kind='bar')
plt.title('Fraud Rate by Hour')
plt.ylabel('fraud_rate')
plt.show()

fraud_by_hour.sort_values(ascending=False).head(5)
```



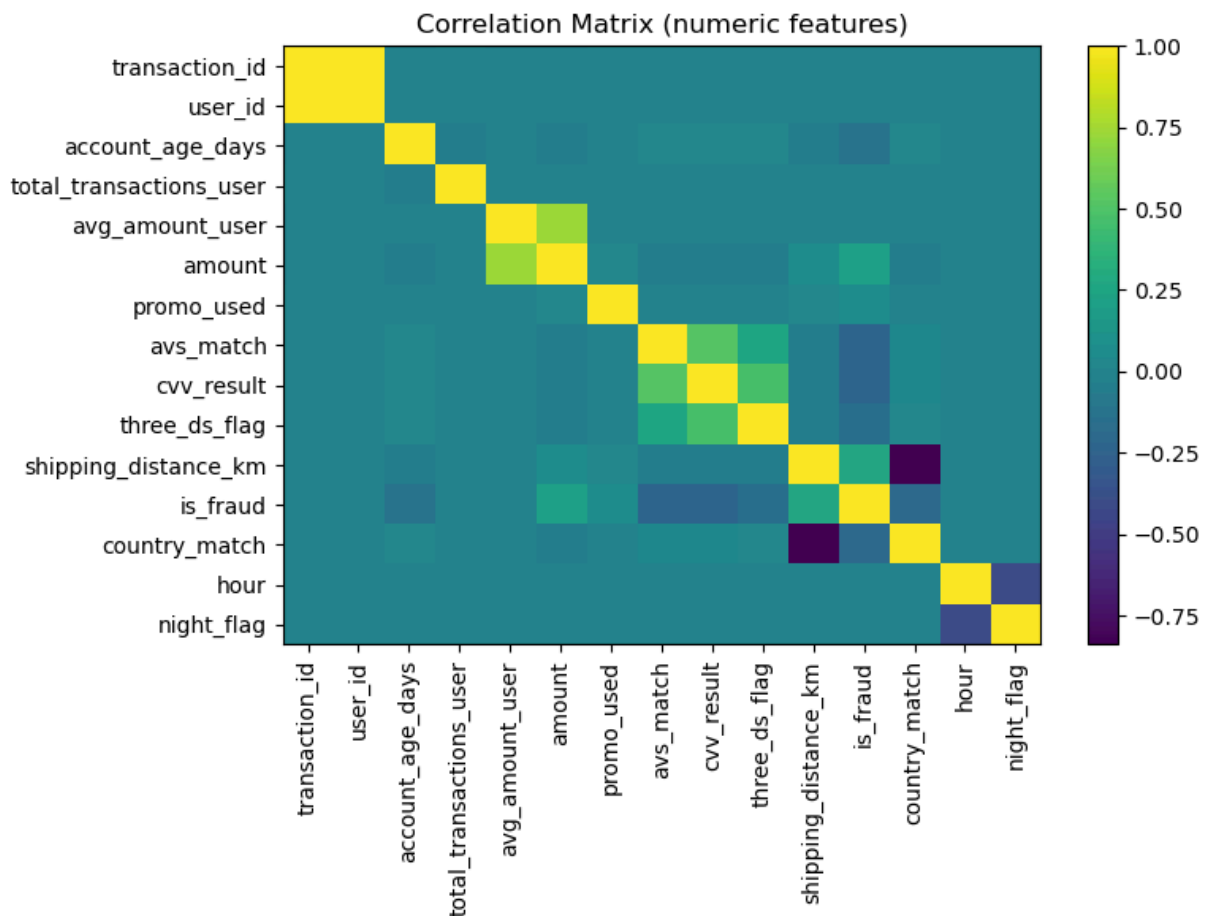


```
Out[6]: hour
23      0.025071
1       0.024208
2       0.023928
8       0.023670
16      0.023121
Name: is_fraud, dtype: float64
```

```
In [7]: # Numeric-only correlation
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
corr = df[num_cols].corr()

fig = plt.figure(figsize=(8,6))
plt.imshow(corr, aspect='auto')
plt.colorbar()
plt.xticks(range(len(num_cols)), num_cols, rotation=90)
plt.yticks(range(len(num_cols)), num_cols)
plt.title('Correlation Matrix (numeric features)')
plt.tight_layout()
plt.show()

corr.loc[['is_fraud'], :].T.sort_values('is_fraud', ascending=False).head(10)
```



Out[7]:

	is_fraud
is_fraud	1.000000
shipping_distance_km	0.270004
amount	0.202499
promo_used	0.067351
night_flag	0.002114
avg_amount_user	-0.001236
hour	-0.001911
user_id	-0.005294
transaction_id	-0.005295
total_transactions_user	-0.017093

In [8]: *# Optional: prepare features for modeling (no training here to keep it EDA-1*

```
drop_cols = ['is_fraud', 'transaction_id', 'transaction_time']
X = df.drop(columns=drop_cols, errors='ignore').select_dtypes(include=[np.nu
y = df['is_fraud'].astype(int)

# First split off test (20%), then split remaining into train (75%) / val (2
X_temp, X_test, y_temp, y_test = train_test_split(
```

```

    X, y, test_size=0.2, stratify=y, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
)

print("Train size:", X_train.shape, " Fraud rate:", y_train.mean())
print("Val size: ", X_val.shape, " Fraud rate:", y_val.mean())
print("Test size: ", X_test.shape, " Fraud rate:", y_test.mean())

```

Train size: (179817, 13) Fraud rate: 0.022066879104867727
Val size: (59939, 13) Fraud rate: 0.022055756685964063
Test size: (59939, 13) Fraud rate: 0.022055756685964063

In [9]: *# — Step 1: Feature Scaling —*
Fit ONLY on train to prevent data leakage, then transform val & test

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)
X_test_scaled  = scaler.transform(X_test)

print("Features scaled ✓")
print(f" Train shape : {X_train_scaled.shape}")
print(f" Val shape   : {X_val_scaled.shape}")
print(f" Test shape  : {X_test_scaled.shape}")
print(f"\nFeature list : {list(X_train.columns)}")

```

Features scaled ✓
Train shape : (179817, 13)
Val shape : (59939, 13)
Test shape : (59939, 13)

Feature list : ['user_id', 'account_age_days', 'total_transactions_user', 'avg_amount_user', 'amount', 'promo_used', 'avs_match', 'cvv_result', 'three_d_s_flag', 'shipping_distance_km', 'country_match', 'hour', 'night_flag']

In [10]: *# — Step 2: Train Baseline – Logistic Regression —*
class_weight='balanced' compensates for 2.2% fraud without SMOTE (Week 2)

```

lr_baseline = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    solver='lbfgs',
    random_state=42
)
lr_baseline.fit(X_train_scaled, y_train)

print("Logistic Regression (baseline) trained ✓")
print(f" Iterations ran : {lr_baseline.n_iter_[0]}")
print(f" Classes       : {lr_baseline.classes_}")

```

Logistic Regression (baseline) trained ✓
Iterations ran : 13
Classes : [0 1]


```
In [11]: # — Step 3: Evaluate on Validation Set —————

y_val_pred = lr_baseline.predict(X_val_scaled)
y_val_proba = lr_baseline.predict_proba(X_val_scaled)[:, 1]

print("=" * 50)
print("  Validation Set – Classification Report")
print("=" * 50)
print(classification_report(y_val, y_val_pred, target_names=['Legit', 'Fraud']))
print(f"ROC-AUC Score      : {roc_auc_score(y_val, y_val_proba):.4f}")
print(f"Avg Precision Score: {average_precision_score(y_val, y_val_proba):.4f}")

=====
Validation Set – Classification Report
=====
```

	precision	recall	f1-score	support
Legit	0.9958	0.8818	0.9353	58617
Fraud	0.1372	0.8336	0.2356	1322
accuracy			0.8807	59939
macro avg	0.5665	0.8577	0.5855	59939
weighted avg	0.9768	0.8807	0.9199	59939

```
ROC-AUC Score      : 0.9292
Avg Precision Score: 0.5615
```

```
In [12]: # — Step 4: Confusion Matrix —————

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

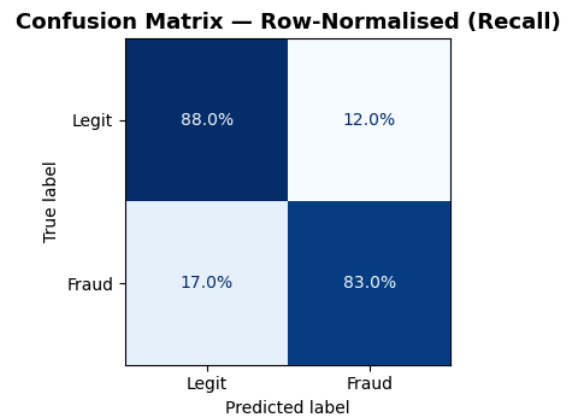
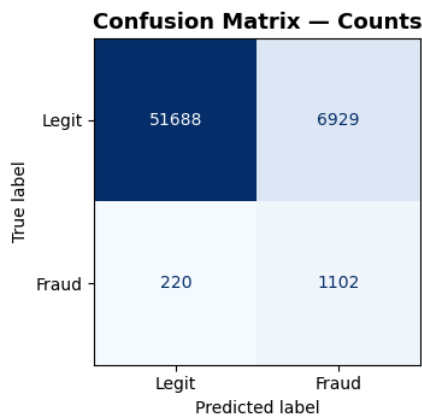
# Raw counts
cm = confusion_matrix(y_val, y_val_pred)
ConfusionMatrixDisplay(cm, display_labels=['Legit', 'Fraud']).plot(
    ax=axes[0], colorbar=False, cmap='Blues'
)
axes[0].set_title('Confusion Matrix – Counts', fontsize=13, fontweight='bold')

# Row-normalised (recall per class)
cm_norm = confusion_matrix(y_val, y_val_pred, normalize='true')
disp = ConfusionMatrixDisplay(cm_norm, display_labels=['Legit', 'Fraud'])
disp.plot(ax=axes[1], colorbar=False, cmap='Blues')
for text in disp.text_.ravel():
    text.set_text(f"{float(text.get_text()):.1%}")
axes[1].set_title('Confusion Matrix – Row-Normalised (Recall)', fontsize=13, fontweight='bold')

plt.suptitle('Baseline Logistic Regression – Validation Set', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives (Legit → Legit) : {tn:,}")
print(f"False Positives (Legit → Fraud) : {fp:,}")
print(f"False Negatives (Fraud → Legit) : {fn:,} ← missed frauds")
print(f"True Positives (Fraud → Fraud) : {tp:,} ← caught frauds")
```

Baseline Logistic Regression — Validation Set



True Negatives (Legit → Legit) : 51,688
 False Positives (Legit → Fraud) : 6,929
 False Negatives (Fraud → Legit) : 220 ← missed frauds
 True Positives (Fraud → Fraud) : 1,102 ← caught frauds

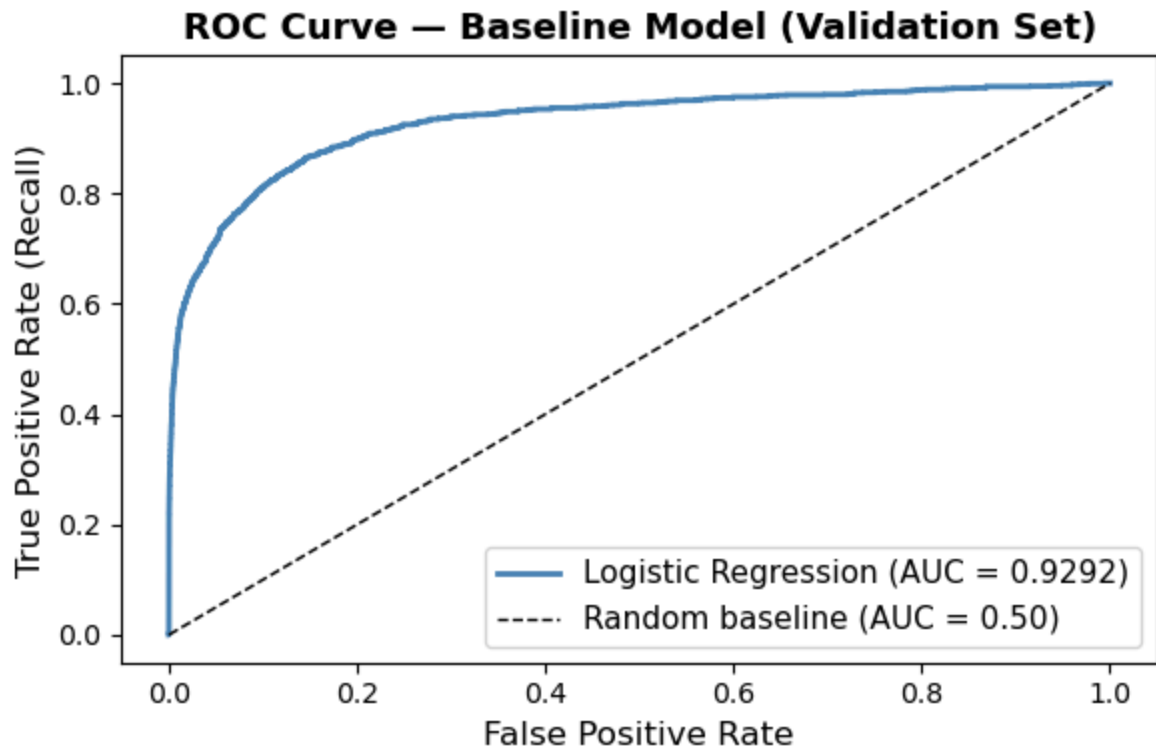
```
In [13]: # — Step 5: ROC Curve + Baseline Summary Table —————

fpr, tpr, _ = roc_curve(y_val, y_val_proba)
auc = roc_auc_score(y_val, y_val_proba)

fig = plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, color='steelblue', lw=2, label=f'Logistic Regression (AUC
plt.plot([0, 1], [0, 1], 'k--', lw=1, label='Random baseline (AUC = 0.50)')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title('ROC Curve — Baseline Model (Validation Set)', fontsize=13, fontwe
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()

# — Summary table — benchmark for all future models —————
baseline_summary = pd.DataFrame([
    'Model' : 'Logistic Regression (baseline)',
    'Accuracy' : round(accuracy_score(y_val, y_val_pred), 4),
    'Precision(Fraud)' : round(precision_score(y_val, y_val_pred), 4),
    'Recall(Fraud)' : round(recall_score(y_val, y_val_pred), 4),
    'F1(Fraud)' : round(f1_score(y_val, y_val_pred), 4),
    'ROC-AUC' : round(roc_auc_score(y_val, y_val_proba), 4),
    'Avg Precision' : round(average_precision_score(y_val, y_val_proba),
])).set_index('Model')

print("=" * 60)
print(" Baseline Performance Summary (Validation Set)")
print("=" * 60)
baseline_summary
```



=====

Baseline Performance Summary (Validation Set)

=====

Out [13]:

	Accuracy	Precision(Fraud)	Recall(Fraud)	F1(Fraud)	ROC-AUC	Avg Precision
Model						
Logistic Regression (baseline)	0.8807	0.1372	0.8336	0.2356	0.9292	0.5615

```
In [14]: # — Step 6: Seller-Level Risk Aggregation —————
# Uses the validation set predictions from Logistic Regression baseline
# In production this would run on incoming transactions in real time

import pandas as pd
import numpy as np

# — 6a: Attach predicted probabilities back to validation rows —————
# X_val still has the original index from the full dataframe
val_df = X_val.copy()
val_df['is_fraud_actual'] = y_val.values
val_df['fraud_prob'] = y_val_proba # already computed in Step 5
val_df['predicted_fraud'] = y_val_pred # already computed in Step 5

# — 6b: AVG_RISK_SCORE per user —————
# Rolling mean of predicted fraud probability per user
# Represents: how risky is this seller on average across all their transactions

avg_risk = (
    val_df.groupby('user_id')['fraud_prob']
```

```

        .mean()
        .rename('avg_risk_score')
        .round(4)
    )

# — 6c: HIGH_RISK_FRACTION per user —————
# Fraction of transactions where predicted probability exceeds threshold (0.
# Threshold of 0.3 is intentionally lower than 0.5 to be conservative

RISK_THRESHOLD = 0.3

high_risk_fraction = (
    val_df.groupby('user_id')['fraud_prob']
    .apply(lambda x: (x >= RISK_THRESHOLD).mean())
    .rename('high_risk_fraction')
    .round(4)
)

# — 6d: RISK_TREND_SLOPE per user —————
# Linear slope of fraud_prob over transaction sequence per user
# Positive slope = fraud risk is increasing over time (escalating behavior)
# Negative slope = risk is decreasing
# Requires at least 2 transactions per user to compute

from numpy.polynomial.polynomial import polyfit

def compute_slope(probs):
    if len(probs) < 2:
        return 0.0
    x = np.arange(len(probs))
    # polyfit returns [intercept, slope] for degree=1
    slope = polyfit(x, probs.values, 1)[1]
    return round(float(slope), 6)

risk_trend_slope = (
    val_df.groupby('user_id')['fraud_prob']
    .apply(compute_slope)
    .rename('risk_trend_slope')
)

# — 6e: Combine into Seller Risk Profile —————
seller_risk_profile = pd.concat(
    [avg_risk, high_risk_fraction, risk_trend_slope],
    axis=1
)

# Add transaction count per user for context
seller_risk_profile['tx_count'] = (
    val_df.groupby('user_id')['fraud_prob'].count()
)

# Add actual fraud rate per user (ground truth – for validation only)
seller_risk_profile['actual_fraud_rate'] = (
    val_df.groupby('user_id')['is_fraud_actual'].mean().round(4)
)

```

```
print(f"Seller risk profiles computed for {len(seller_risk_profile)} users")
print(seller_risk_profile.describe())
seller_risk_profile.head(10)
```

Seller risk profiles computed for 6000 users

	avg_risk_score	high_risk_fraction	risk_trend_slope	tx_count
count	6000.000000	6000.000000	6000.000000	6000.000000
mean	0.220088	0.227912	-0.000355	9.989833
std	0.120219	0.179204	0.038102	3.073104
min	0.022300	0.000000	-0.370687	1.000000
25%	0.130175	0.100000	-0.014290	8.000000
50%	0.200400	0.200000	-0.000168	10.000000
75%	0.285650	0.333300	0.013719	12.000000
max	0.947700	1.000000	0.478820	21.000000

	actual_fraud_rate
count	6000.000000
mean	0.022413
std	0.084013
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

Out [14]:

	avg_risk_score	high_risk_fraction	risk_trend_slope	tx_count	actual_fraud_
--	----------------	--------------------	------------------	----------	---------------

user_id					
1	0.3372	0.3000	0.011490	10	
2	0.0711	0.0000	-0.002644	9	
3	0.1283	0.0000	-0.005344	7	
4	0.2199	0.2000	-0.006846	10	
5	0.4468	0.5000	0.212227	4	
6	0.2072	0.1111	-0.047024	9	
7	0.3281	0.5000	-0.013000	12	
8	0.1603	0.0000	0.015567	6	
9	0.2120	0.1667	0.015354	12	
10	0.1155	0.1667	-0.002623	12	

In [15]:

```
# — Step 7: Flag and Rank High-Risk Sellers —————
# A seller is HIGH RISK if they meet ANY of these criteria:
#   - avg_risk_score      >= 0.40  (consistently elevated probability)
#   - high_risk_fraction >= 0.20  (20%+ of their transactions are suspicious)
#   - risk_trend_slope   >= 0.01  (fraud risk is actively accelerating)

HIGH_RISK_AVG      = 0.40
HIGH_RISK_FRACTION = 0.20
HIGH_RISK_SLOPE    = 0.01
```

```

seller_risk_profile['is_high_risk'] = (
    (seller_risk_profile['avg_risk_score'] >= HIGH_RISK_AVG) |
    (seller_risk_profile['high_risk_fraction'] >= HIGH_RISK_FRACTION) |
    (seller_risk_profile['risk_trend_slope'] >= HIGH_RISK_SLOPE)
)

high_risk_sellers = (
    seller_risk_profile[seller_risk_profile['is_high_risk']]
    .sort_values('avg_risk_score', ascending=False)
)

print(f"Total sellers in validation : {len(seller_risk_profile)}")
print(f"High-risk sellers flagged : {len(high_risk_sellers)}")
print(f"Flag rate : {len(high_risk_sellers)/len(seller_risk_profile)}")
print()
print("Top 10 Highest Risk Sellers:")
print(high_risk_sellers.head(10).to_string())

```

```

Total sellers in validation : 6000
High-risk sellers flagged : 3823
Flag rate : 63.72%

```

Top 10 Highest Risk Sellers:

user_id	avg_risk_score	high_risk_fraction	risk_trend_slope	tx_count	actual_fraud_rate	is_high_risk
4392	0.9477	1.0000	0.005209	5	1.0000	True
2956	0.9320	1.0000	-0.023008	7	0.8571	True
4271	0.8899	1.0000	-0.004452	7	0.4286	True
3087	0.8207	1.0000	-0.168045	4	0.7500	True
1724	0.8184	0.9000	-0.056282	10	0.7000	True
4444	0.8063	1.0000	0.001337	11	0.6364	True
2303	0.8052	1.0000	0.009600	10	0.7000	True
93	0.8032	0.8571	-0.003325	7	0.4286	True
3910	0.7853	0.7500	-0.252317	4	0.7500	True
2632	0.7783	0.8889	0.022092	9	0.5556	True

In [16]: *# — Step 8: Validate Seller Risk Scores Against Actual Fraud Rates —*
This is how we prove the seller layer is meaningful –
high-risk sellers should have significantly higher actual fraud rates

```

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Plot 1: avg_risk_score vs actual_fraud_rate

```

```

axes[0].scatter(
    seller_risk_profile['avg_risk_score'],
    seller_risk_profile['actual_fraud_rate'],
    alpha=0.4, s=10, color='steelblue'
)
axes[0].set_xlabel('Avg Risk Score (predicted)')
axes[0].set_ylabel('Actual Fraud Rate')
axes[0].set_title('Avg Risk Score vs Actual Fraud Rate')

corr0 = seller_risk_profile[['avg_risk_score', 'actual_fraud_rate']].corr().i
axes[0].text(0.05, 0.95, f'r = {corr0:.3f}',
             transform=axes[0].transAxes, fontsize=11, color='red')

# Plot 2: high_risk_fraction vs actual_fraud_rate
axes[1].scatter(
    seller_risk_profile['high_risk_fraction'],
    seller_risk_profile['actual_fraud_rate'],
    alpha=0.4, s=10, color='darkorange'
)
axes[1].set_xlabel('High Risk Fraction')
axes[1].set_ylabel('Actual Fraud Rate')
axes[1].set_title('High Risk Fraction vs Actual Fraud Rate')

corr1 = seller_risk_profile[['high_risk_fraction', 'actual_fraud_rate']].corr
axes[1].text(0.05, 0.95, f'r = {corr1:.3f}',
             transform=axes[1].transAxes, fontsize=11, color='red')

# Plot 3: High-risk vs low-risk seller actual fraud rate distribution
axes[2].boxplot(
    [
        seller_risk_profile[seller_risk_profile['is_high_risk']]['actual_fra
        seller_risk_profile[~seller_risk_profile['is_high_risk']]['actual_fr
    ],
    labels=['High-Risk Sellers', 'Low-Risk Sellers'],
    patch_artist=True,
    boxprops=dict(facecolor='salmon')
)
axes[2].set_ylabel('Actual Fraud Rate')
axes[2].set_title('Actual Fraud Rate: High vs Low Risk Sellers')

plt.suptitle('Seller Risk Profile Validation', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Correlation summary
print("Correlation with actual fraud rate:")
print(f"  avg_risk_score      : {corr0:.4f}")
print(f"  high_risk_fraction: {corr1:.4f}")

```

```

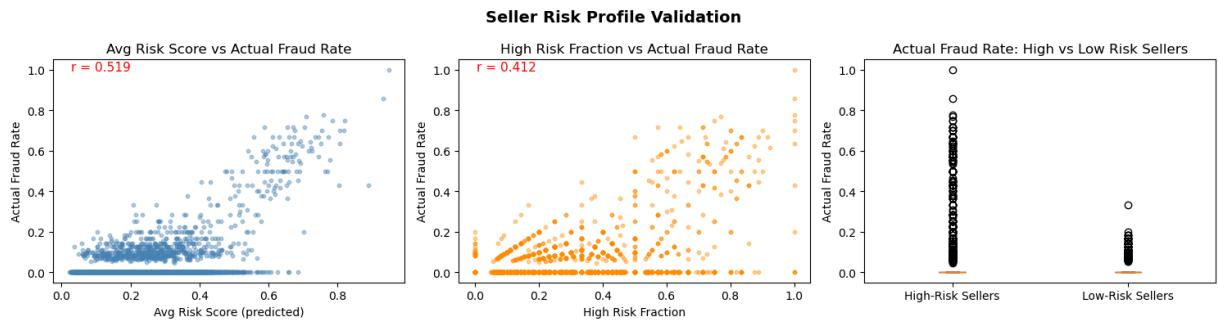
/var/folders/n0/8c1pt88x1rjdjkmkt4tyfmmw0000gn/T/ipykernel_34741/720585726.p
y:38: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has
been renamed 'tick_labels' since Matplotlib 3.9; support for the old name wi
ll be dropped in 3.11.

```

```

axes[2].boxplot(

```



Correlation with actual fraud rate:

avg_risk_score : 0.5195

high_risk_fraction: 0.4117

In []: